SEMANA 16 · ARQUITECTURA EN LA NUBE

IA en la Nube III: Capacidades de los Modelos y Repaso de la Unidad 3

Parte 1 de 4 · Razonamiento Avanzado, Planificación y Modelos Reflexivos

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

Del Reconocimiento de Patrones al Razonamiento

Diferencias estructurales entre la predicción asociativa directa y los procesos de deliberación interna

Modelo Clásico (Sistema 1 · Respuesta Rápida)

Mecanismo Operativo:
Predicción autorregresiva de siguiente token sin deliberación intermedia.

Comportamiento en Problemas Complejos:
Alucinación en cálculos matemáticos y fallas en lógica multi-etapa.

Tiempo de Inferencia:
Estrictamente proporcional a la longitud del texto devuelto.

Dominio de Aplicación:
Redacción fluida, traducción, síntesis de texto y atención conversacional.

Modelo de Razonamiento (Sistema 2 · Deliberativo)

Mecanismo Operativo:
Generación de cadena de pensamiento interna con auto-verificación.

Comportamiento en Problemas Complejos:
Descompone en sub-metas, prueba hipótesis y retrocede si falla.

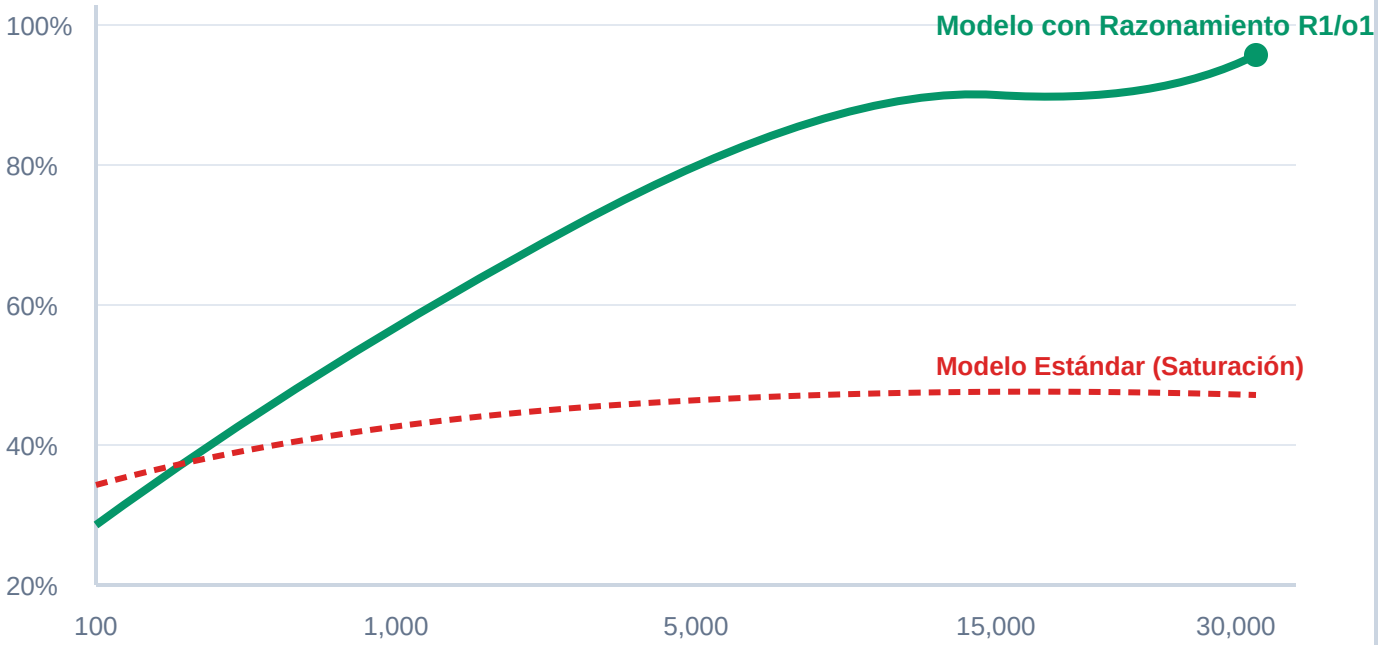
Tiempo de Inferencia:
Mayor latencia inicial por producción intensiva de tokens ocultos.

Dominio de Aplicación:
Matemáticas avanzadas, refactorización de software y auditoría forense.

Fuente: DeepSeek-R1 Whitepaper & OpenAI o1 Architecture Overview · Consulta 2026-11

Escalado del Cómputo en Tiempo de Prueba

Inversión de ciclos de GPU en la inferencia para elevar exponencialmente la precisión del modelo



Nuevo Paradigma de Cómputo

Inversión en Inferencia:
Mayor número de tokens de pensamiento eleva linealmente el éxito en benchmarks.

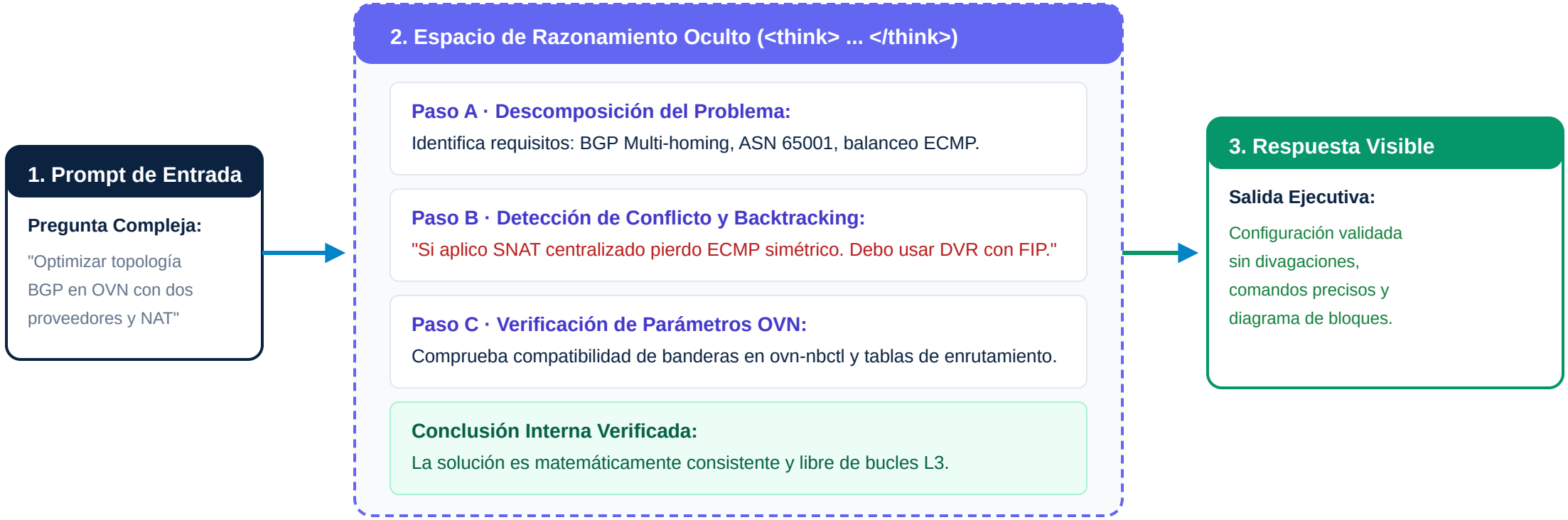
Equilibrio Eficiente:
Un modelo de 8B con razonamiento puede superar a uno de 70B estándar.

Desafío para la Nube:
Peticiones que duran 30 a 60 segundos; obliga a rediseñar timeouts HTTP.

Fuente: Scaling LLM Test-Time Compute Optimally · Snorkel AI & UC Berkeley Research 2024

Cadena de Pensamiento Oculta

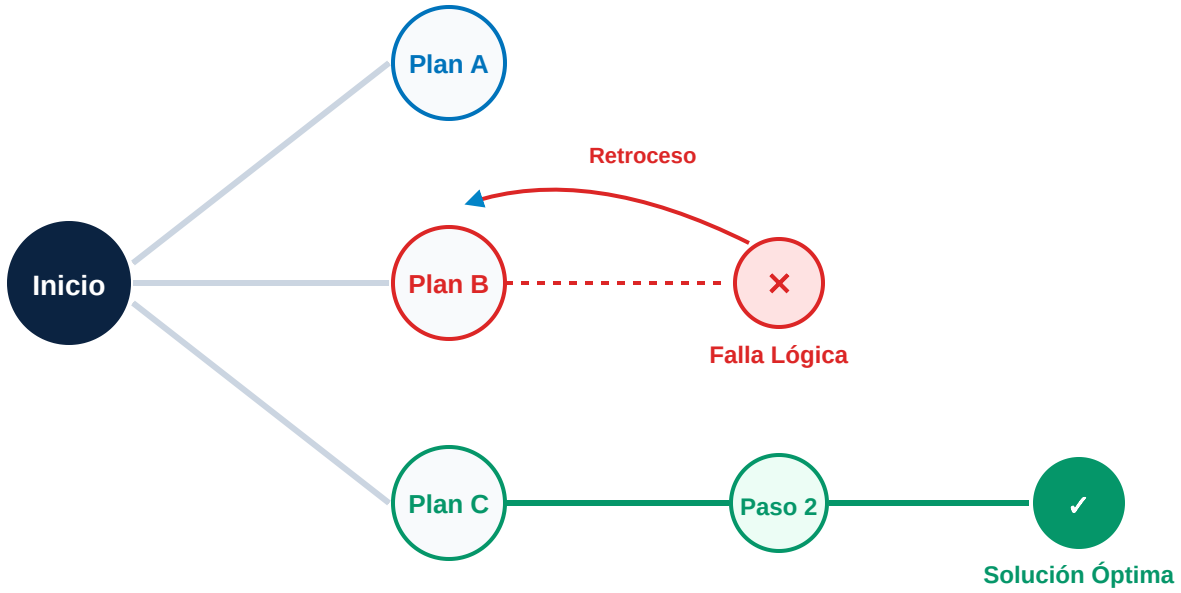
Aislamiento entre los tokens de deliberación interna y la respuesta final entregada al usuario



Fuente: DeepSeek-R1 Architecture & OpenAI API Reasoning Tokens Specification

Árboles de Pensamiento y Backtracking

Exploración sistemática de ramas de solución con evaluación heurística y retroceso ante errores



Mecanismos del Algoritmo ToT

1. Generación de Pensamientos:

Explora múltiples ramificaciones en lugar de comprometerse con una sola secuencia lineal.

2. Poda y Retroceso (Backtracking):

Cuando un paso produce una contradicción, el árbol poda esa rama y regresa al nodo anterior.

3. Búsqueda en Amplitud / Profundidad:

Combina BFS o Monte Carlo Tree Search para maximizar la probabilidad de hallar el óptimo.

Fuente: Tree of Thoughts: Deliberate Problem Solving with Large Language Models · Yao et al. 2023

Entrenamiento sin Supervisión Humana: GRPO

Incentivación del razonamiento autónomo mediante Group Relative Policy Optimization en DeepSeek-R1-Zero

Modelo Base Puro

DeepSeek-V3 Base

Sin fine-tuning supervisado

Sin ejemplos humanos CoT

Inicio de Bucle RL

Muestreo Grupal (Group Sampling)

Respuesta y1: Genera razonamiento breve

Resultado incorrecto · Recompensa: 0

Respuesta y2: Razona y se auto-corrige

Resultado correcto · Recompensa: +1.0

Respuesta y3: Razonamiento circular fallido

Falla formato · Recompensa: -0.5

Calcula ventaja relativa del grupo sin modelo crítico

Recompensas Basadas en Reglas

1. Recompensa de Exactitud (Rule-based):

Validación matemática en compilador o pruebas unitarias. ¡Cero sesgo o subjetividad humana!

2. Recompensa de Formato:

Exige encuadrar el pensamiento dentro de <think> y la respuesta final en el bloque exterior.

Fenómeno Emergente ("Aha Moment"):

El modelo aprende espontáneamente a reflexionar, re-evaluar y corregirse para maximizar el premio.

Fuente: DeepSeek-R1 Technical Report · Section 2: Pure RL on Base Model

Verificación Determinista y Recompensas

Sistemas de evaluación formal mediante compiladores, pruebas unitarias y motores simbólicos

1. Dominio Matemático Simbólico

Verificador Formal:

Sistemas como SymPy o Lean 4 validan demostraciones paso a paso con certeza binaria.

Naturaleza de la Recompensa:

1.0 si la demostración es válida,
0.0 ante cualquier fallo de equivalencia.

Elimina la necesidad de supervisión humana en benchmarks AIME y GSM8K.

2. Ejecución y Pruebas de Código

Entorno de Pruebas Unitarias:

Compilador GCC / intérprete Python ejecuta suites de pruebas automatizadas con assert.

Naturaleza de la Recompensa:

Proporcional al porcentaje de pruebas unitarias aprobadas (pass rate unitario).

Base del éxito en HumanEval y resolución de issues en SWE-bench.

3. Límites en Dominios Abiertos

Ausencia de Verificador:

Redacción literaria, estrategia de negocios y diseño gráfico carecen de verdad objetiva.

Riesgo de Recompensa (Reward Hacking):

El modelo aprende trucos retóricos para complacer al evaluador sin rigor analítico.

Por ello el razonamiento puro brilla en código, matemáticas y arquitecturas cloud.

Fuente: Verifiable Rewards in Reinforcement Learning for Code and Math · OpenAI & DeepSeek Studies

Anatomía de los Tokens de Razonamiento

Manejo de cuotas, costos de facturación y ocultamiento de trazas en APIs corporativas

POST /v1/chat/completions (Respuesta HTTP)

```
{
   "id": "chatcmpl-9xR1model",
   "model": "deepseek-reasoner / o1",
   "choices": [{
      "message": {
         "reasoning_content": "Paso 1: Validar...",
         "content": "Solución arquitectónica final..."
      }
   }],
   "usage": {
      "prompt_tokens": 150,
      "reasoning_tokens": 4200,
      "completion_tokens": 4480
   }}
```

Reglas de Facturación y Seguridad

1. Facturación de Tokens Ocultos:

Los reasoning_tokens se cobran como tokens de salida al precio completo de generación, aunque no se muestren.

2. Aislamiento y Redacción de Trazas:

Proveedores como OpenAI no entregan el texto bruto de razonamiento para proteger la propiedad intelectual.

3. Modelos Abiertos (DeepSeek-R1):

Entrega el campo reasoning_content completo, permitiendo auditar cada deducción lógica en servidores propios.

Fuente: OpenAI Platform Docs: Reasoning Tokens & DeepSeek-R1 API Specifications

Costos y Retardos en Producción

Impacto de la deliberación profunda en el tiempo al primer token y presupuestos de nube

Impacto en Latencia (Tiempo de Espera)

Modelo Estándar (Claude 3.5 Sonnet / GPT-4o):

TTFT: 0.8 s · Duración total de respuesta: 2.5 a 4.0 s

Modelo con Razonamiento Profundo (o1 / R1):

TTFT: 12.0 s a 45.0 s (deliberación silenciosa)

Requiere indicadores de carga o arquitecturas asíncronas.

Riesgo en Balanceadores de Carga:

Los timeouts estándar de 30s en ALB/Nginx abortan la conexión antes de que el modelo termine de pensar.

Impacto en Costos de Nube

Multiplicador de Tokens:

Una consulta simple puede consumir 15x a 30x más tokens

Costo Efectivo por Solución Verificada:

Aunque el costo unitario por llamada es más alto, evita ciclos fallidos de reintento en tareas de código.

Recomendación Arquitectónica:

Nunca conectar modelos de razonamiento a chat síncrono para preguntas triviales; aislar en colas SQS asíncronas.

Fuente: Latency and Throughput Benchmarks for Reasoning LLMs · Artificial Analysis 2026

Control de Presupuesto de Razonamiento

Parámetros de limitación para acotar el esfuerzo cognitivo según la criticidad de la petición

Nivel Bajo (reasoning_effort: low)

Tokens de Pensamiento:

Límite: ~500 a 1,500 tokens

Latencia típica: 2 a 5 segundos

Casos Recomendados:

Consultas SQL sencillas, revisión gramatical técnica y FAQs.

Ahorro de costo: 70% vs máximo

Nivel Medio (reasoning_effort: medium)

Tokens de Pensamiento:

Límite: ~3,000 a 8,000 tokens

Latencia típica: 8 a 15 segundos

Casos Recomendados:

Scripts de automatización bash/python, análisis de logs y manifiestos K8s.

Balance estándar para ingeniería

Nivel Alto (reasoning_effort: high)

Tokens de Pensamiento:

Límite: Hasta 25,000+ tokens

Latencia típica: 30 a 90 segundos

Casos Recomendados:

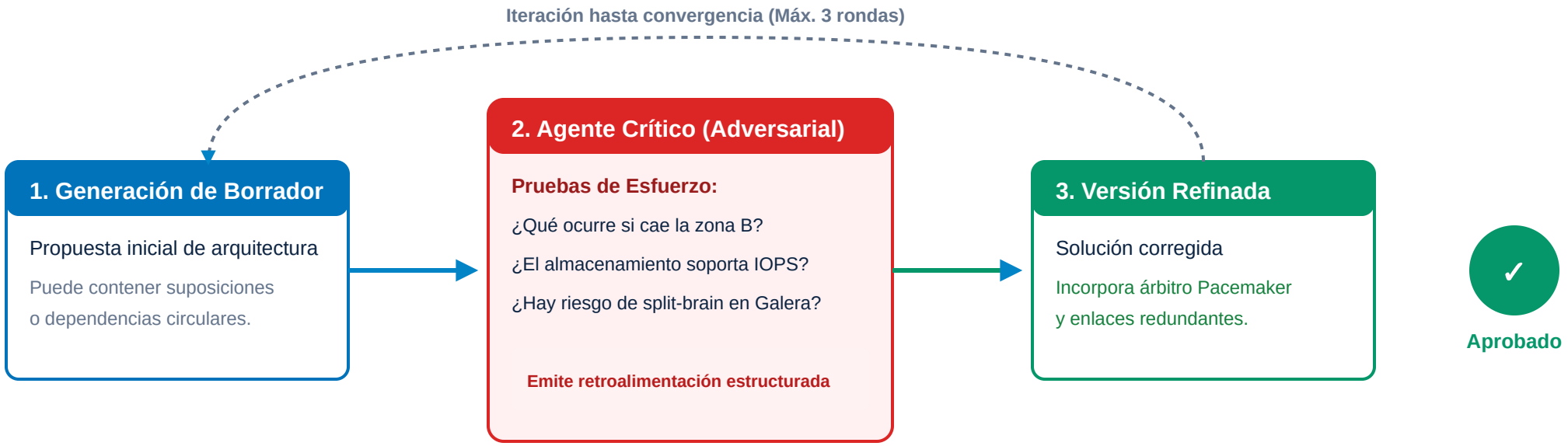
Auditorías criptográficas, resolución de vulnerabilidades de día cero.

Máximo poder deliberativo

Fuente: OpenAI Reasoning Effort Parameter Specification & DeepSeek Max Tokens Guide

Auto-Corrección y Debate Multi-Agente

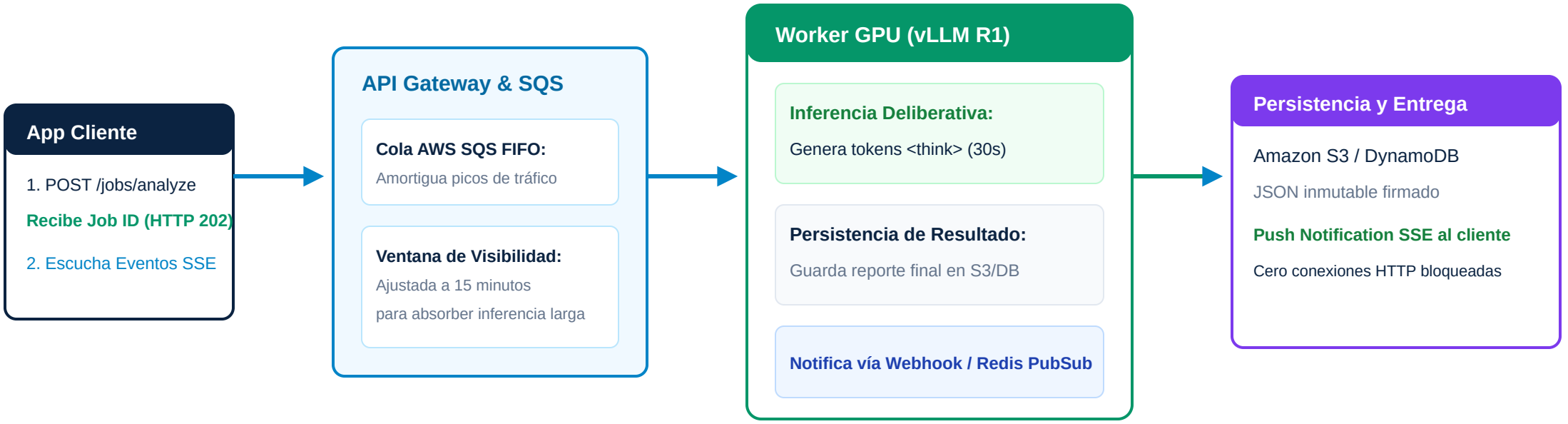
Técnicas de refinamiento iterativo y contrastación dialéctica para eliminar alucinaciones lógicas



Fuente: Self-Refine: Iterative Refinement with Self-Feedback · Madaan et al. 2023

Arquitectura de Servicio para Modelos R1/o1

Patrones asíncronos con colas desacopladas y Server-Sent Events para absorber retardos de deliberación



Fuente: Asynchronous LLM Architecture Patterns · AWS Architecture Center

CRITERIO DE SELECCIÓN

¿Cuándo Usar Razonamiento Profundo?

Matriz de idoneidad entre modelos de pensamiento intensivo frente a modelos tradicionales

Tipo de Tarea de Negocio	Modelo Idóneo	Justificación Técnica	Impacto en TCO
Refactorización y Depuración de Código	**Razonamiento (R1 / o1)**	Prueba mental de casos límite y sintaxis	Ahorro en reintentos
Atención al Cliente / Chatbot Web	**Estándar (Haiku / 8B)**	Prioridad en TTFT < 1s y lenguaje empático	Bajo costo por token
Modelado Financiero y Actuarial	**Razonamiento (R1 / o1)**	Cálculos numéricos estrictos sin error	Costo alto justificado
Resumen y Clasificación Documental	**Estándar (Claude Sonnet / Llama 70B)**	Comprensión contextual sin lógica profunda	Excelente relación
Auditoría de Vulnerabilidades Cloud	**Razonamiento (R1 / o1)**	Rastreo de rutas de escalada de privilegios	Crítico para seguridad

Regla Práctica del Arquitecto:

Reservar los modelos de razonamiento para tareas donde el costo de un error supere ampliamente el costo del cómputo adicional.

Fuente: Decision Matrix for Reasoning Model Workloads · AWS & Cloud Industry Standards

Diagnóstico Industrial en Planta Mamonal

Caso ilustrativo empresarial (no es una empresa real) · Corporación Industrial del Caribe (Barranquilla)

1. Alerta Telemetría SCADA

Anomalía Detectada:

Caída de presión de 15% en reactores químicos y alza de vibración en bombas.

Riesgo Operativo:

Parada no programada de planta con pérdidas de miles de dólares por hora.

Despacho Asíncrono:

Sensor envía snapshot de telemetría a cola de análisis en nodo OpenStack propio.

2. Razonamiento con R1 Local

Deliberación Interna (4,500 tokens):

Cruza curvas termodinámicas con bitácoras de mantenimiento de los últimos 6 meses.

Detección de Causa Raíz:

Descarta fallo de motor; identifica cavitación por válvula de alivio V-402 descalibrada.

Soberanía de Datos:

Inferencia ejecutada en clúster local; ningún parámetro sale de la planta industrial.

3. Remediación en Terreno

Procedimiento al Operador:

Guía paso a paso con torque específico y secuencia de cierre seguro de compuertas.

Tiempo Total de Respuesta:

28 segundos de análisis exhaustivo; evita intervención humana a ciegas.

Resultado Comprobado:

Presión normalizada en 12 minutos; incidente cerrado sin parar la línea química.

Buenas Prácticas para Razonamiento

Directrices de ingeniería para gobernar cargas de inferencia deliberativa en la nube

1. Desacoplamiento Asíncrono Obligatorio

Nunca invocar modelos de razonamiento en la ruta síncrona HTTP:

Emplear colas SQS, almacenamiento de resultados en S3 y entrega push mediante WebSockets o Server-Sent Events (SSE) para evitar caídas de proxy.

Regla: Timeouts de cola configurados a un mínimo de 15 minutos.

2. Control Estricto de Presupuesto Cognitivo

Fijar reasoning_effort o max_thinking_tokens por categoría:

No conceder cuotas infinitas de tokens a los usuarios. Mapear niveles bajos para tareas rutinarias y niveles altos únicamente para operaciones críticas.

Regla: Auditoría diaria de reasoning_tokens en CloudWatch.

3. Verificación Determinista en el Bucle

Integrar compiladores y suites de pruebas en la salida:

Aprovechar que el razonamiento produce artefactos formales (código o SQL) para validarlos automáticamente en sandboxes antes de aplicarlos en producción.

Regla: Cero ejecuciones de código sin paso previo por sandbox KVM.

4. Ruteo Híbrido Inteligente

Filtrar con modelos ligeros antes de invocar razonamiento:

El 80% de las consultas corporativas no justifican la latencia ni el costo de o1 o R1. Despachar a Claude 3 Haiku o Llama 8B salvo requerimiento expreso.

Regla: Reducir el TCO global desviando el tráfico rutinario.

SEMANA 16 · ARQUITECTURA EN LA NUBE

IA en la Nube III: Capacidades de los Modelos y Repaso de la Unidad 3

Parte 2 de 4 · Programación, Generación de Código y Sandboxing en la Nube

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

Evolución de los Modelos de Código

De simples asistentes de autocompletado sintáctico a agentes de refactorización autónoma

Gen 1: Autocompletado (2021-2022)

Capacidad Principal:

Sugerencias línea a línea dentro del IDE.

Limitación Crítica:

Cero visibilidad del proyecto completo; incapaz de validar llamadas a librerías.

Modelos: Codex inicial, Tabnine

Gen 2: Chat & Explicación (2023)

Capacidad Principal:

Explicación de código, traducción y snippets.

Limitación Crítica:

El desarrollador debe copiar y pegar a mano; sin ejecución ni pruebas automáticas.

Modelos: GPT-4, Claude 2, CodeLlama

Gen 3: Agentes de Código (2024-2026)

Capacidad Principal:

Navegación del repo, parches multi-archivo y tests.

Innovación en Ejecución:

Ejecuta bash, compila, corre suites de prueba y se auto-corrige hasta pasar el CI/CD.

Modelos: Claude 3.5 Sonnet, o1, DeepSeek-R1

Fuente: The State of AI in Software Engineering · ACM Queue & GitHub Research 2026

EVALUACIÓN RIGUROSA

Benchmarks: HumanEval vs SWE-bench

Diferencia entre aprobar funciones sintéticas aisladas y resolver incidencias reales de GitHub

HumanEval (Benchmark Sintético · 164 Problemas)

Alcance de la Tarea:
Escribir el cuerpo de una función en un archivo único aislado.

Sesgo y Saturación:
Saturado (>90% pass@1); alta contaminación en el preentrenamiento.

Realismo en Producción:
Muy bajo: no mide capacidad de navegar bases de código complejas.

SWE-bench Verified (Estándar de la Industria)

Alcance de la Tarea:
Resolver incidencias reales de GitHub en Django, SymPy, scikit-learn.

Dificultad Rigurosa:
Requiere clonar repositorios, reproducir fallos y aprobar pytest real.

Líderes de Mercado:
Claude 3.5 Sonnet (~50%), OpenAI o1 (~48%), DeepSeek-R1 (~49%).

Fuente: SWE-bench Verified Benchmark & HumanEval Leaderboard · Consulta 2026-11

Claude 3.5 Sonnet para Código

Capacidades de razonamiento estructural, comprensión de grafos de dependencias y emisión de parches unificados

1. Contexto de 200k Tokens

Visión de Repositorio:

Ingesta simultánea de decenas de archivos clave, interfaces y dependencias.

Evita la pérdida de contexto entre módulos distantes.

2. Diffs Estructurados

Formato Git Unificado:

Genera parches precisos con líneas + y - exactas, sin reescribir todo el archivo.

Ahorro radical en tokens de salida y tiempo de GPU.

3. Invocación de Herramientas

Tool Use Nativo:

Llama a grep, find, cat y pytest con esquemas JSON estrictos y deterministas.

Permite inspección quirúrgica antes de realizar cambios.

4. Baja Alucinación en APIs

Fidelidad Sintáctica:

No inventa parámetros ni métodos inexistentes en librerías cloud (boto3, k8s).

Asegura compatibilidad con versiones exactas de SDKs.

Fuente: Anthropic Research: Claude 3.5 Sonnet System Capabilities · Consulta 2026-11

El Riesgo de la Ejecución Arbitraria

Vulnerabilidades de inyección de comandos, extracción de variables de entorno y escape de aislamiento

Vector 1: Código Inseguro Generado

Inyección de Comandos:

El modelo genera subprocesos sin validar entradas: os.system("rm -rf " + path).

Fuga de Credenciales:

Scripts que leen os.environ o ~/.aws/credentials y transmiten secretos a endpoints remotos.

Vector 2: Escape de Contenedor

Docker Compartiendo Kernel:

Los contenedores tradicionales comparten el kernel del host; un exploit expone el nodo.

Consumo Abusivo de CPU/RAM:

Bucles infinitos o ataques de denegación de servicio (DoS) por bombas lógicas.

Principio de Seguridad Cero

Aislamiento por Hardware:

Nunca ejecutar código de IA en el host ni en contenedores sin virtualización real.

Entornos Desechables:

Cada ejecución debe arrancar en una VM limpia y destruirse al terminar.

Fuente: OWASP Top 10 for Large Language Model Applications · LLM02: Sensitive Info & Insecure Code

Sandboxing con AWS Firecracker

Micro-máquinas virtuales KVM con arranque en 5 milisegundos y consumo base de 5 megabytes

MicroVM 1 (Sesión Agente A)

Kernel Propio Dedicado:
Linux guest kernel minimalista sin drivers legacy

Aislamiento de Memoria:
5 MB RAM base · Arranque en ~5 ms

Red: Dispositivo virtio-net sin salida a Internet

MicroVM 2 (Sesión Agente B)

Kernel Propio Dedicado:
Aislamiento por virtualización de CPU (VT-x)

Almacenamiento Efímero:
virtio-block en RAM disk destruido al cerrar

Red: Loopback local exclusivamente

MicroVM 3 (Sesión Agente C)

Kernel Propio Dedicado:
Garantía de no interferencia multinquilino

Límites de Cómputo:
Máximo 2 vCPUs y 512 MB por sandbox

Control: Timeout duro de ejecución: 10 segundos

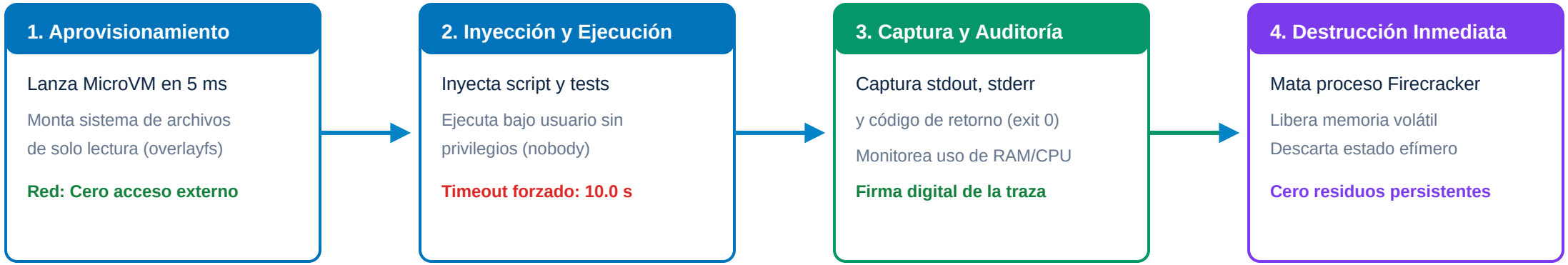
Servidor Físico Bare-Metal · Linux Host Kernel (/dev/kvm)

Control de acceso seccomp, cgroups v2, jailer enjaulamiento chroot

Fuente: Firecracker: Lightweight Virtualization for Serverless Applications · USENIX NSDI

Arquitectura de un Sandbox Cloud Seguro

Ciclo de vida de ejecución efímera con aislamiento de red, cuotas de cómputo y auditoría forense



Fuente: Secure Code Execution Sandboxes in the Cloud · Security Engineering Best Practices

Generación Determinista de Pruebas

El patrón de co-generación de código y suites unitarias con assertions matemáticas

Módulo de Implementación (calcular_cuotas.py)

```python
def calcular_cuota_tpm(usuarios, tpm_base=500):
    # Valida concurrencia y reserva buffer 20%
    if usuarios <= 0:
        raise ValueError("Usuarios deben ser > 0")
    bruto = usuarios * tpm_base
    return int(bruto * 1.20)
```

Suite de Pruebas Unitarias (test_cuotas.py)

```python
import pytest

def test_caso_estandar():
    assert calcular_cuota_tpm(10) == 6000

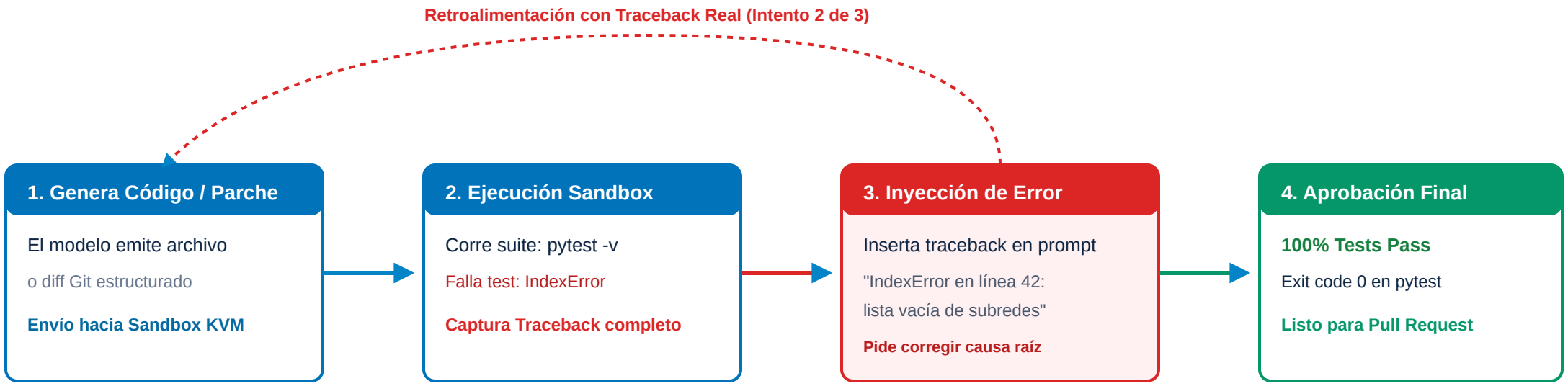
def test_caso_invalido():
    with pytest.raises(ValueError):
        calcular_cuota_tpm(0)

def test_caso_borde():
    assert calcular_cuota_tpm(1) == 600
```


Fuente: Test-Driven Development with AI Agents · IEEE Software 2025

Depuración Asistida y Auto-Corrección

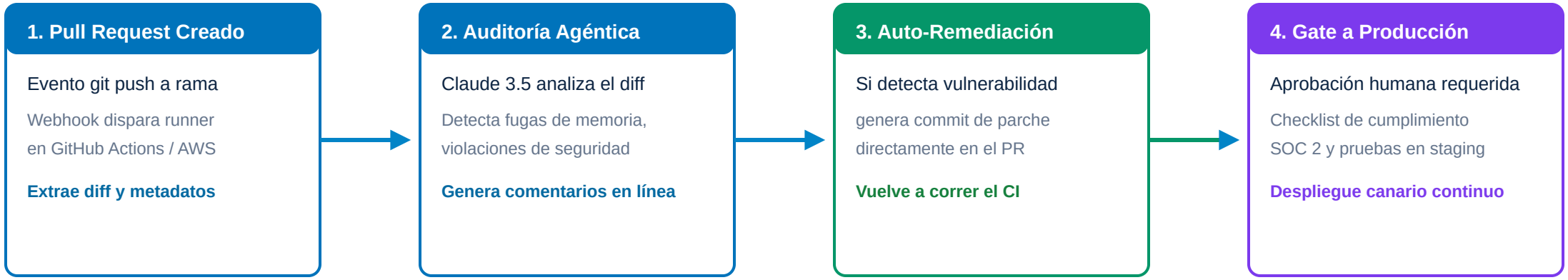
El bucle iterativo de compilación, captura de trazas de excepción y corrección de sintaxis



Fuente: Agentic Coding Loops and Error Recovery Mechanisms · SWE-bench Technical Report

Integración en Tuberías CI/CD

Incrustación de agentes evaluadores en GitHub Actions y AWS CodePipeline con guardrails de despliegue



Fuente: GitHub Actions CI/CD Integration Patterns with Autonomous AI Reviewers 2026

Escaneo de Seguridad SAST en Código de IA

Detección de vulnerabilidades de inyección SQL, hardcoded secrets y deserialización insegura

1. Credenciales Incrustadas

Riesgo Habitual:

El modelo genera ejemplos con tokens reales o llaves privadas ficticias (AKIA...).

Regla Semgrep SAST:

Bloquea cualquier string que coincida con patrones de credenciales; exige AWS Secrets.

CWE-798: Uso de credenciales cableadas

2. Inyecciones SQL / NoSQL

Riesgo Habitual:

Concatenación de strings en sentencias SELECT en lugar de usar prepared statements.

Regla Semgrep SAST:

Exige el uso mandatorio de parámetros enlazados o mapeadores como SQLAlchemy.

CWE-89: Inyección de comandos SQL

3. Paquetes Alucinados

Riesgo de Typosquatting:

El modelo importa librerías ficticias en pip/npm que un atacante puede registrar.

Regla de Verificación:

Auditoría con pip-audit contra el índice oficial; bloqueo de paquetes sin reputación previa.

Ataque de envenenamiento de cadena de suministro

Ecosistema de Herramientas Agénticas

Comparativa entre interfaces IDE integradas, CLI de terminal y entornos de agentes autónomos

Herramienta Agéntica	Entorno Operativo	Mecanismo de Control	Caso de Uso Óptimo
Claude Code (Terminal CLI)	Consola Bash / SSH directa	Bucle interactivo con confirmación	Refactorización masiva de repos
Aider (Git Pair Programming)	Terminal vinculada a Git	Commits atómicos automáticos	Desarrollo feature-by-feature
Cursor / Windsurf (Fork VS Code)	IDE nativo visual	Indexación vectorial del espacio	Programación diaria interactiva
GitHub Copilot Workspace	Nube / Entorno Web GitHub	De Issue a Pull Request guiado	Resolución de tickets de backlog

Convergencia Tecnológica:

Todas las herramientas convergen hacia el mismo núcleo: modelos de razonamiento (Claude 3.5 / o1) interactuando con árboles de sintaxis abstracta (AST) e invocando sandboxes para validar el código antes de mostrarlo al ingeniero.

Fuente: Agentic Developer Tooling Ecosystem Landscape · Developer Experience Review 2026

Propiedad Intelectual y Licenciamiento

Riesgo de contaminación por licencias virales (GPL) y auditoría de procedencia en código de IA

1. Contaminación Copyleft (GPL)

Mecanismo de Riesgo:

El modelo memoriza fragmentos literales de código licenciado bajo GPL v2/v3.

Consecuencia Corporativa:

Efecto viral: obligación legal de liberar el código propietario de la organización.

Auditoría con Black Duck / FOSSA

2. Indemnización de Nube

Compromiso Contractual:

AWS Bedrock y Microsoft ofrecen cláusulas de indemnización por copyright.

Condición Exigida:

Activar filtros de similitud de código y usar modelos oficiales sin alteraciones.

Blindaje legal frente a demandas de terceros

3. Política de Gobernanza

Filtro de Código Público:

Bloquear sugerencias que coincidan en más de 150 caracteres con repositorios públicos.

Atribución y Trazabilidad:

Registrar en metadatos de Git qué código fue asistido por IA y validado por humanos.

Garantía de debida diligencia corporativa

CASO ILUSTRATIVO DE NEGOCIO

Migración Automatizada de Microservicios

Caso ilustrativo empresarial (no es una empresa real) · Corporación Industrial del Caribe (Barranquilla)

1. Deuda Técnica Legacy

Estado Inicial:

24 microservicios en Python 3.8 con librerías obsoletas y fallas de seguridad.

Estimación Tradicional:

4 meses de trabajo manual para un equipo de 3 ingenieros de software.

Paralización de nuevos features de negocio

2. Pipeline Agéntico en Nube

Orquestación Claude 3.5:

Agente actualiza sintaxis a Python 3.12 y migra de requests sincrónico a httpx.

Validación Firecracker:

Ejecuta 850 pruebas unitarias en paralelo; auto-corrige fallos de tipado con mypy.

Ejecución en 72 horas continuas

3. Resultados Verificados

Métricas de Éxito:

100% microservicios migrados exitosamente; cero regresiones en ambiente staging.

Eficiencia y Ahorro:

Reducción del 85% en tiempo de entrega; el equipo se concentró en la arquitectura.

Despliegue exitoso en clúster Kubernetes

PRINCIPIOS DE INGENIERÍA

Buenas Prácticas en Código con IA

Directrices de rigor técnico para gobernar el desarrollo asistido por modelos en la nube

1. Aislamiento Estricto en Sandboxes

Nunca ejecutar código de IA en estaciones o hosts de producción:

Toda prueba debe correr en microVMs Firecracker efímeras sin salida a red, con memoria restringida y timeouts estrictos de máximo diez segundos.

Regla: Virtualización por hardware como barrera infranqueable.

2. Co-Generación Mandatoria de Pruebas

Ningún código se considera aceptable sin suite unitaria:

Exigir al modelo redactar tests exhaustivos cubriendo casos estándar, escenarios de borde y excepciones. La aprobación requiere 100% de éxito.

Regla: Evaluación determinista con pytest antes de mostrar el código.

3. Escaneo SAST y Validación de Dependencias

Auditar cada línea contra vulnerabilidades OWASP:

Escanear el diff con Semgrep para impedir credenciales incrustadas, inyecciones SQL y verificar que los paquetes importados existan en PyPI.

Regla: Cero paquetes alucinados en las dependencias.

4. El Ingeniero como Revisor Crítico Final

La responsabilidad legal y técnica nunca se delega:

El rol del ingeniero evoluciona de escribir sintaxis rutinaria a auditar diseño, mantenibilidad, licencias de propiedad intelectual y seguridad.

Regla: Todo merge a producción requiere firma de un humano.

SEMANA 16 · ARQUITECTURA EN LA NUBE

IA en la Nube III: Capacidades de los Modelos y Repaso de la Unidad 3

Parte 3 de 4 · Delegación, Sistemas Agénticos y Protocolos

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

De Modelos Aislados a Agentes Autónomos

Transición de la generación de texto estático a la interacción interactiva con el entorno de nube

Modelo Aislado Tradicional (Oráculo)

Interacción:
Un único turno prompt-respuesta sin contacto con el mundo exterior.

Estado del Sistema:
Incapaz de verificar el estado real de un servidor, IP o base de datos.

Carga Operativa:
El operador humano debe ejecutar cada comando y copiar los resultados.

Limitación:
Conocimiento congelado en la fecha de corte de su preentrenamiento.

Agente Autónomo en la Nube

Interacción:
Bucle continuo de razonamiento, invocación de herramientas y observación.

Acceso a Infraestructura:
Consulta APIs de AWS, OpenStack, métricas Prometheus y bases de datos.

Autonomía:
Resuelve objetivos descomponiendo tareas sin supervisión paso a paso.

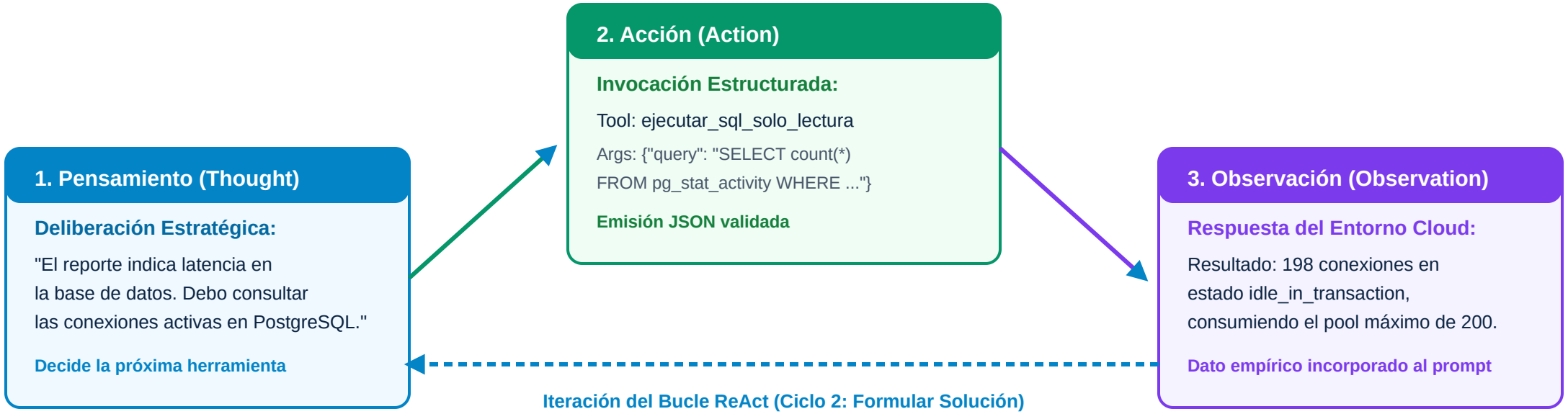
Memoria Persistente:
Mantiene historial episódico y consulta documentación viva mediante RAG.

Fuente: The Anatomy of Autonomous AI Agents in Cloud Environments · ACM Computing Surveys 2026

PATRÓN OPERATIVO

El Patrón ReAct: Pensar y Actuar

Sinergia iterativa entre deliberación lógica (Thought), invocación de APIs (Action) y lectura del entorno (Observation)



Fuente: ReAct: Synergizing Reasoning and Acting in Language Models · Yao et al. ICLR 2023

Invocación de Herramientas y Function Calling

Esquemas declarativos JSON-Schema para garantizar tipado estricto y validación determinista

Definición de Tool (JSON-Schema Declarativo)

```
{
  "name": "reiniciar_servicio_openstack",
  "description": "Reinicia un demonio de cómputo en Nova",
  "parameters": {
    "type": "object",
    "properties": {
      "servicio": {"type": "string", "enum": ["nova", "ovn"]},
      "nodo": {"type": "string", "pattern": "^comp-[0-9]$"},
      "motivo": {"type": "string", "minLength": 10}
    },
    "required": ["servicio", "nodo", "motivo"]
  }}
```

Mecanismo de Ejecución Segura

1. Salida Forzada Estructurada:

El modelo no puede inventar argumentos libres; el decodificador CUDA fuerza que los tokens cumplan el esquema gramatical exacto.

2. Validación de Tipos del Lado Servidor:

Pydantic / Zod valida expresiones regulares y valores enum antes de despachar la llamada al clúster de producción.

3. Control de Acceso RBAC:

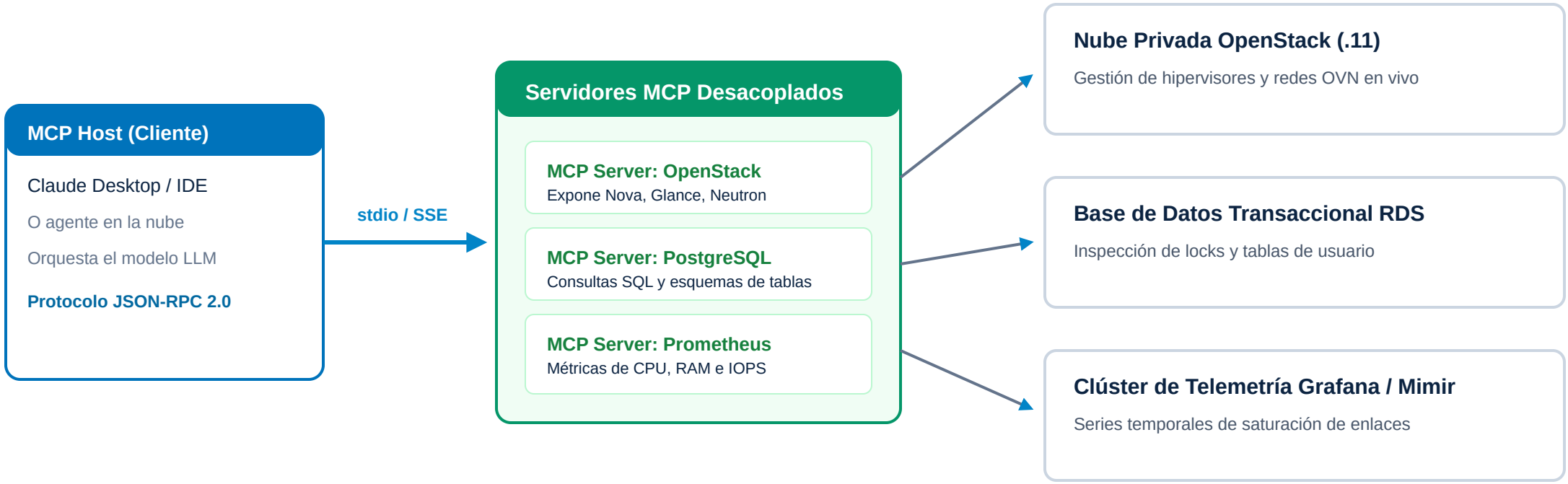
Cada tool opera bajo un rol IAM restringido; operaciones destructivas exigen aprobación humana en dos pasos.

Fuente: OpenAI Function Calling & Anthropic Tool Use Specifications · Guía Oficial 2026

ESTÁNDAR DE INTEGRACIÓN

Model Context Protocol (MCP)

El protocolo abierto creado por Anthropic para desacoplar modelos de fuentes de datos y herramientas



Fuente: Model Context Protocol (MCP) Specification · Anthropic Open Source Standards 2026

Recursos, Prompts y Herramientas en MCP

Las tres primitivas estandarizadas que definen el contrato entre clientes y servidores del protocolo

1. Resources (Recursos Pasivos)

Definición y Rol:

Datos de solo lectura estructurados bajo un identificador URI: file://, db://, logs://.

Ejemplo en la Nube:

openstack://instances/active/summary
Retorna texto o binario sin efectos secundarios.

Suscripción activa: notifica cambios en tiempo real

2. Prompts (Plantillas Guiadas)

Definición y Rol:

Flujos de trabajo predefinidos por el servidor para asistir al usuario en tareas complejas.

Ejemplo en la Nube:

prompt: "auditoria_seguridad_openstack"
Inyecta checklist y reglas CIS automáticamente.

Reutilización estándar de conocimiento experto

3. Tools (Operaciones con Efecto)

Definición y Rol:

Funciones ejecutables por el modelo que modifican estado o invocan cómputo externo.

Ejemplo en la Nube:

tool: "migrar_instancia_nova"
Dispara migración en vivo entre hipervisores.

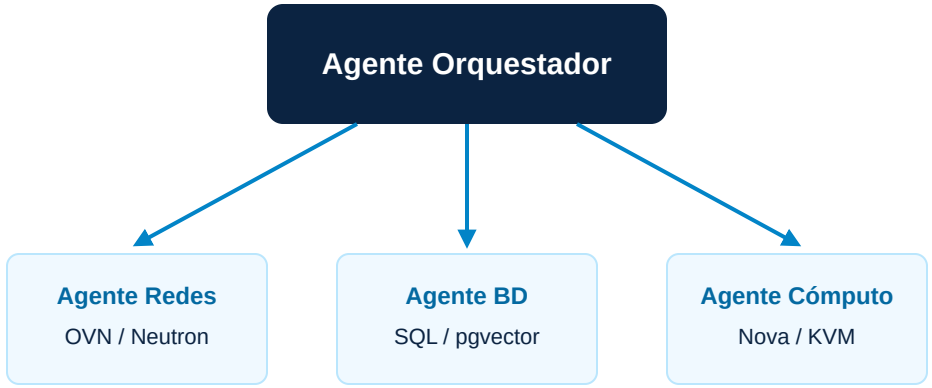
Requiere control estricto de autorizaciones

Fuente: Model Context Protocol Specification: Core Primitives · Anthropic Docs 2026

Orquestador Central vs Enjambre (Swarm)

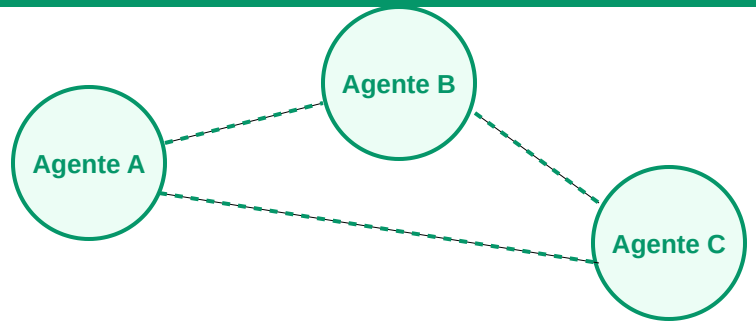
Patrones arquitectónicos para la coordinación y delegación de trabajo entre múltiples agentes

Patrón 1: Orquestador Centralizado (Hub-and-Spoke)



Ventaja:
Control estricto, flujo determinista, fácil auditoría y depuración.

Patrón 2: Enjambre Descentralizado (Swarm)



Riesgo en Producción:
Comunicación no determinista, riesgo de bucles de mensajes infinitos.

Fuente: Multi-Agent Collaboration Patterns: Orchestrator vs Swarm · OpenAI & AutoGen 2026

Compactación del Contexto de Trabajo

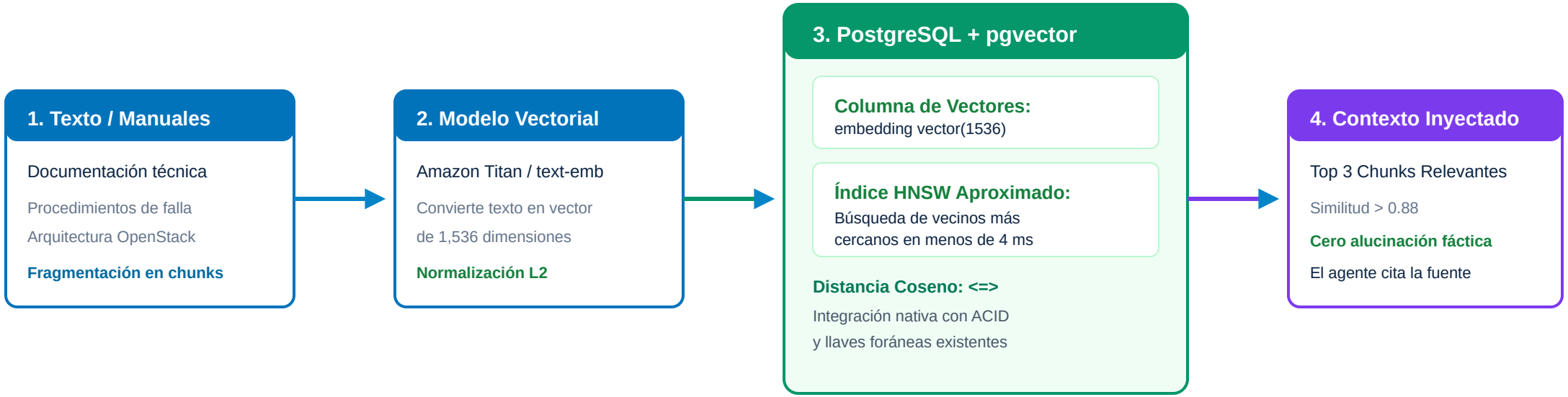
Técnicas de destilación de historial y poda de tokens para evitar la saturación de la memoria a corto plazo



Fuente: Context Compression and Pruning Techniques for Long Conversations · Anthropic 2026

Memoria Semántica con Embeddings

Indexación vectorial con pgvector en PostgreSQL sobre AWS RDS para recuperación de largo plazo



Fuente: PostgreSQL pgvector Extension Documentation & Vector Search in Cloud Databases 2026

ANÁLISIS COMPARATIVO

Ventanas Masivas de Contexto vs RAG

Evaluación multidimensional entre arquitecturas de 2 millones de tokens y recuperación híbrida

Criterio Arquitectónico	Ventana Masiva (1M - 2M Tokens)	Arquitectura RAG Vectorial Híbrida
Latencia al Primer Token (TTFT)	**Muy Alta (15 a 45 segundos)**	**Ultra-Baja (200 a 600 ms)**
Costo Marginal por Consulta	$3.00 a $6.00 USD por llamada	< $0.005 USD (Ahorro del 99%)
Recuperación de Datos Específicos	Degradación en el centro (Lost in the middle)	**Alta precisión por ranking vectorial**
Síntesis Holística Multi-Documental	**Excelente (Ve todo el libro o código a la vez)**	Limitada por fragmentación de chunks

Recomendación de Arquitectura de Producción (Patrón Híbrido):

Utilizar RAG para filtrar los 5 a 10 documentos más relevantes del repositorio empresarial reduciendo el contexto a ~32k tokens, y despachar ese bloque filtrado al modelo. Se obtiene la máxima velocidad, bajo costo y cero degradación analítica.

Fuente: Needles in Haystacks: Empirical Comparison of Long-Context vs RAG Architectures 2026

Mínimo Privilegio y Blast Radius

Límites estrictos en roles de ejecución IAM y confirmación humana en el bucle para acciones destructivas

Zona 1: Solo Lectura (Autónomo)

Permisos Otorgados:

DescribeInstances, GetMetricData, SELECT de solo lectura en réplicas.

Mecanismo de Ejecución:

El agente ejecuta en bucle sin pausa; cero riesgo de alterar infraestructura.

Blast radius: Cero impacto operacional

Zona 2: Cambios No Destructivos

Permisos Otorgados:

Crear snapshot EBS, autoescalar pods, crear ramas temporales en Git.

Mecanismo de Ejecución:

Autónomo con registro de auditoría en AWS CloudTrail con retención 1 año.

Blast radius: Bajo y completamente reversible

Zona 3: Acciones Críticas (HITL)

Operaciones Restringidas:

TerminateInstances, DROP TABLE, modificar Security Groups de producción.

Human-in-the-Loop Obligatorio:

El agente genera el plan de cambio; requiere aprobación manual con 2FA.

Blast radius: Controlado por intervención humana

Fuente: Least Privilege Architecture for Autonomous Cloud Agents · Cloud Security Alliance 2026

Prevención de Bucles y Estancamiento

Guardrails para detectar ciclos infinitos de herramientas y límites de gasto por sesión

1. Límite de Iteraciones (Max Iterations)

Tope forzado de vueltas en el bucle ReAct:

Si el agente alcanza 25 iteraciones sin converger a una solución,
el runtime aborta el bucle y escala la incidencia a un operador humano.

Regla: max_iterations = 25 como cortafuegos operativo.

2. Detección de Llamadas Repetitivas

Detección de estancamiento (Stuck Loop Detection):

Un interceptor compara el hash de los argumentos invocados.
Si la misma herramienta falla 3 veces con idénticos parámetros, se bloquea.

Regla: Romper el ciclo inyectando aviso de estancamiento en el prompt.

3. Presupuesto Monetario por Sesión

Tope de gasto financiero acumulado en dólares:

Cada sesión tiene asignado un límite máximo (ej. $5.00 USD).
Si el consumo de tokens supera el tope, se corta la invocación al modelo.

Regla: Hard budget limit gestionado en pasarela LiteLLM.

4. Temporizador de Inactividad (Watchdog)

Monitoreo de procesos zombis en tareas de fondo:

Si un subproceso o microVM de sandbox no responde en 60 segundos,
un guardián supervisor envía una señal SIGKILL y libera recursos.

Regla: Cero procesos huérfanos consumiendo memoria del nodo.

Fuente: Guardrails and Circuit Breakers for Autonomous Agent Runtimes 2026

Observabilidad con OpenTelemetry

Trazabilidad de grafos de razonamiento, latencia de herramientas y métricas de inferencia

Traza Distribuida OpenTelemetry (Span Tree · Trace ID: 4bf92f3577b34da6)

```
Root: agente_diagnostico_cloud (Duración: 14.8s · Tokens: 6,420 · Costo: $0.042 USD)
  ├── Span 1: llm.call (model: claude-3-5-sonnet · prompt: 1,200 · comp: 480 · 2.4s)
  │   ├── Span 2: tool.call (server: mcp-openstack · method: get_ports · 0.6s)
  │   └── Span 3: db.vector (table: kb · cosine · 4ms)
  └── Span 4: llm.call (model: claude-3-5-sonnet · prompt: 3,400 · comp: 1,340 · 3.8s)
```

Métricas Prometheus:

agent_iterations_total · agent_cost_dollars

Calidad de Respuesta:

RAG groundness score: 0.94 · Cero alucinación

Auditoría de Seguridad:

Registro inmutable en S3 de cada acción ejecutada

Fuente: OpenTelemetry Semantic Conventions for Generative AI & LangSmith Tracing 2026

CASO ILUSTRATIVO DE NEGOCIO

Auditor Agéntico de OpenStack

Caso ilustrativo empresarial (no es una empresa real) · Corporación Industrial del Caribe (Barranquilla)

1. Desafío Operacional

Infraestructura Dispersa:

Clúster OpenStack con 12 hipervisores, 340 máquinas virtuales y 80 redes OVN.

Problema:

IPs flotantes huérfanas, volúmenes Cinder sin adjuntar y puertos zombies.

Desperdicio del 22% de capacidad útil

2. Despliegue Agente MCP

Conectividad Segura:

Agente opera mediante servidor MCP local conectado con tokens Fernet a Keystone.

Inspección Nocturna:

Ejecuta 18 iteraciones ReAct cruzando registros de auditoría con tráfico conntrack.

Cero llamadas al exterior; 100% privado

3. Ahorro e Impacto Real

Limpieza Validada:

Identificó 42 Floating IPs sin uso y 18 TB de volúmenes huérfanos desasociados.

Aprobación Humana:

El agente emitió un PR con el script de limpieza; el administrador aprobó en 1 clic.

Recuperación de capacidad sin nuevo hardware

Buenas Prácticas en Sistemas Agénticos

Directrices de ingeniería para diseñar arquitecturas multi-agente robustas y seguras

1. Estandarización mediante MCP

Desacoplar herramientas y datos mediante el protocolo abierto:

Implementar servidores MCP para exponer bases de datos, APIs de nube y telemetría. Evitar integraciones propietarias atadas a un único LLM.

Regla: El protocolo sobrevive a cualquier cambio de proveedor de modelo.

2. Principio de Menor Privilegio (RBAC)

Delimitar con rigor el radio de impacto de cada herramienta:

Conceder solo lectura por defecto. Acciones mutativas requieren roles IAM temporales y operaciones destructivas exigen confirmación humana (HITL).

Regla: Cero tokens de administración broad en manos del agente.

3. Cortafuegos de Bucles y Presupuesto

Establecer límites duros de ejecución en el runtime:

Fijar un máximo de 25 iteraciones, límite de gasto financiero por sesión y detección de estancamiento para prevenir llamadas repetitivas infinitas.

Regla: Cortafuegos presupuestario inquebrantable.

4. Observabilidad de Extremo a Fin

Instrumentar cada llamada con OpenTelemetry y registros inmutables:

Registrar cada span de razonamiento, latencia de tools y costo en USD. Preservar auditoría forense para cumplir con la regulación de la SIC.

Regla: Toda decisión del agente debe ser reproducible y auditable.

Fuente: Architecture Standards for Enterprise Agentic Systems · Cloud Native Foundation 2026

SEMANA 16 · ARQUITECTURA EN LA NUBE

IA en la Nube III: Capacidades de los Modelos y Repaso de la Unidad 3

Parte 4 de 4 · Repaso Integral de la Unidad 3 y Cierre de Arquitectura en la Nube

Ing. Rodolfo José Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Arquitectura en la Nube · Periodo 2026-2 · Barranquilla, Colombia

VISIÓN HOLÍSTICA

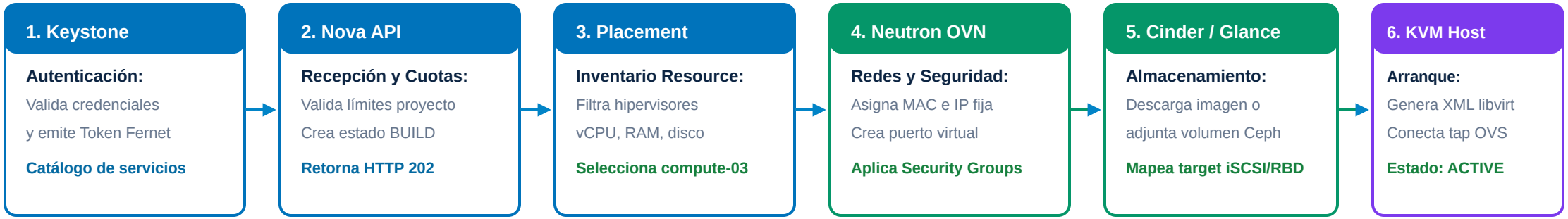
La Pila Tecnológica de la Unidad 3

De los hipervisores bare-metal y redes overlay hacia los modelos de frontera y protocolos agénticos

Capa 4: Agentes y Protocolos Cognitivos	**Model Context Protocol (MCP) · Patrón ReAct · Razonamiento R1/o1** Sandboxing con AWS Firecracker · Co-generación de pruebas · Memoria pgvector
Capa 3: Servicio y Optimización de IA	**Motor vLLM · PagedAttention · Continuous Batching · AWQ 4-bit** AWS Bedrock Serverless · Zero Data Retention (ZDR) · Punto de equilibrio TCO
Capa 2: Cómputo Acelerado y Silicio	**GPUs NVIDIA Hopper (H100) · Tensor Cores · Memoria HBM3 (3.35 TB/s)** Interconexión NVLink (900 GB/s) · RDMA EFA · Instancias AWS p5 / g5
Capa 1: Nube Privada e Infraestructura Base	**OpenStack (Nova, Neutron, Keystone, Glance, Cinder) · Redes OVN Geneve** Alta disponibilidad con Pacemaker, Corosync, Galera MySQL y RabbitMQ Quorum

El Flujo de Creación de una VM

Coordinación coreográfica entre Keystone, Nova, Placement, Glance, Neutron y Cinder



Fuente: OpenStack Architecture Reference Guide · Transactional VM Lifecycle Flow

Redes de Producción: De OVS Clásico a OVN

Superación del cuello de botella centralizado de agentes mediante el plano de control distribuido OVN

Neutron Clásico (ML2 / OVS con iptables)

Enrutamiento Centralizado L3:
Todo el tráfico de Floating IPs y NAT cruza por un nodo de red único.

Seguridad por iptables Legacy:
Retardos de hasta 30s al modificar Security Groups por recarga serial.

Agentes Python Dispersos:
neutron-l3, dhcp y openvswitch-agentent propensos a desincronización.

Neutron Moderno (OVN · Open Virtual Network)

Enrutamiento Totalmente Distribuido (DVR):
Cada hipervisor enruta tráfico L3 y NAT localmente; cero cuellos de botella.

Seguridad Nativa con OVS Conntrack:
Aplicación instantánea en el kernel Linux sin crear puentes veth artificiales.

Bases de Datos OVSDB Ligeras:
Base de datos NB y SB con túneles de encapsulación Geneve.

Fuente: OpenStack Neutron OVN Architecture Guide · Distributed Routing & Geneve Tunnels

Alta Disponibilidad en el Plano de Control

Resiliencia activa mediante clústeres Pacemaker, Corosync, Galera MySQL y RabbitMQ Quorum

Controlador 1 (controller-01)

VIP Pública Virtual (keepalived):
IP Flotante: 200.30.0.10 activa

Galera MySQL (Nodo 1 de 3):
Replicación síncrona multi-maestro (WSREP)

RabbitMQ (Quorum Queue):
Algoritmo de consenso Raft distribuido

Controlador 2 (controller-02)

VIP Pública Virtual (keepalived):
Modo BACKUP en espera pasiva

Galera MySQL (Nodo 2 de 3):
Sincronizado al commit con quórum mayoritario

RabbitMQ (Quorum Queue):
Réplica Raft sincronizada

Controlador 3 (controller-03)

VIP Pública Virtual (keepalived):
Modo BACKUP en espera pasiva

Galera MySQL (Nodo 3 de 3):
Desempata particiones y evita split-brain

RabbitMQ (Quorum Queue):
Garantiza entrega de mensajes ante caídas

Fuente: OpenStack High Availability Guide · Pacemaker, Galera & Quorum Queues

Aceleración de IA: Silicio y Memoria HBM3

Microarquitectura de GPUs, modelo Roofline y contraste entre ancho de banda de memoria y cómputo GEMM

Fase de Prefill (Cómputo Puro · GEMM)

Naturaleza Operativa:

Procesa todo el prompt de entrada de forma masivamente paralela.

Cuello de Botella Dominante:

Compute-bound: limitado por los TFLOPS de los Tensor Cores.

Impacto Arquitectónico:

Determina el Tiempo al Primer Token (TTFT) en la llamada.

Fase de Decode (Memoria · GEMV)

Naturaleza Operativa:

Generación secuencial autorregresiva de salida, token por token.

Cuello de Botella Dominante:

Memory-bound: limitado por el ancho de banda HBM3 (3.35 TB/s).

Impacto Arquitectónico:

Determina el Tiempo por Token de Salida (TPOT) y la fluidez visual.

Fuente: NVIDIA Hopper H100 GPU Architecture Whitepaper & Roofline Performance Modeling

Compresión Matemática: AWQ a 4 Bits

Reducción del 75% en memoria VRAM preservando el 1% de pesos salientes mediante protección por activación

Modelo en Precisión Completa (FP16 · 16 bits)

Consumo de Memoria Llama 70B:
140 GB de VRAM solo en pesos

Requisito de Servidor:
Obliga a desplegar instancias de 2x a 4x GPUs A100 (80GB).

Costo Operativo AWS:
Instancia p4de.24xlarge: ~$29,000 USD mensuales continuos.

Modelo Cuantizado con AWQ (INT4 · 4 bits)

Consumo de Memoria Llama 70B:
38 GB de VRAM (Ahorro del 73%)

Requisito de Servidor:
Ejecuta en una sola GPU NVIDIA A100 (80GB) o 2x RTX 6000 Ada.

Preservación de Calidad:
Retiene más del 99% de precisión en benchmarks MMLU y HumanEval.

Fuente: AWQ: Activation-aware Weight Quantization for LLM Compression · MIT Research

Motores de Alto Rendimiento: vLLM

Gestión de memoria paginada con PagedAttention y servicio de lotes continuo en producción

Innovación 1: PagedAttention

El Problema Tradicional:

Fragmentación externa de memoria; reserva contigua del 80% ocioso.

Solución Paginada:

Trata VRAM como memoria virtual, dividida en bloques de 16 tokens.

Impacto en Producción:

Desperdicio menor al 4%; multiplica por 4x la concurrencia de usuarios.

Innovación 2: Batching Continuo

El Problema Tradicional:

Lotes estáticos: todos deben esperar a que la secuencia más larga termine.

Solución a Nivel de Token:

Inserta nuevas peticiones tan pronto como una secuencia finaliza.

Impacto en Producción:

Throughput saturado de la GPU con colas de espera prácticamente nulas.

Fuente: vLLM: Efficient Memory Management with PagedAttention · UC Berkeley 2023

Gestionado vs Auto-Alojado: Punto de Equilibrio

Curvas financieras de costo total de propiedad: cuándo pagar por token frente a comprar hardware de cómputo

Ruta A: Modelo como Servicio (AWS Bedrock)

Modelo Financiero:

OpEx puro: se factura estrictamente por millón de tokens procesados.

Zona de Rentabilidad Óptima:

Tráfico inferior a 40 millones de tokens diarios o cargas muy variables.

Ventaja Competitiva:

Cero administración de servidores, acceso inmediato a modelos frontera.

Ruta B: Auto-Alojado Propio (vLLM en OpenStack)

Modelo Financiero:

CapEx inicial en servidores o costo fijo horario por instancia de GPU.

Zona de Rentabilidad Óptima:

Tráfico masivo sostenido superior a 40M tokens/día; costo marginal ~0.

Ventaja Competitiva:

Soberanía de datos absoluta, latencia ultra-baja y cero cuotas 429.

Fuente: Total Cost of Ownership Modeling for Cloud AI Architectures 2026

Modelos Cognitivos y Búsqueda ToT

Deliberación en tiempo de prueba mediante tokens de pensamiento oculto y algoritmos de retroceso

1. Escalado en Inferencia

Segunda Ley de Escalado:

Más ciclos de GPU pensando en caliente elevan exponencialmente la exactitud.

Resultado:

Modelos compactos de 8B con R1 superan a modelos de 70B sin pensar.

2. Árboles con Backtracking

Exploración No Lineal:

Genera hipótesis paralelas y poda aquellas que producen contradicciones.

Resultado:

Capacidad de corregirse antes de emitir la primera palabra visible al usuario.

3. Verificación Determinista

Recompensas por Reglas (GRPO):

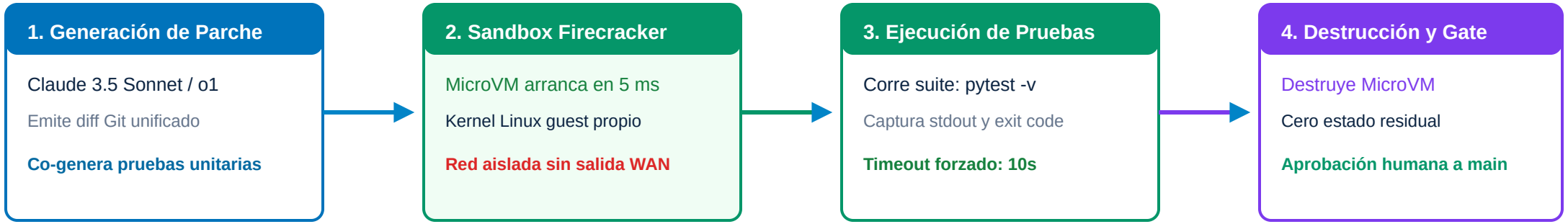
Entrenamiento con compiladores de código y solucionadores matemáticos formales.

Resultado:

Cero sesgo humano; florecimiento de estrategias deductivas comprobables.

Programación Autónoma y Sandboxing KVM

Ejecución segura de código generado mediante microVMs efímeras de AWS Firecracker en la nube



Fuente: Cloud Sandboxing Architecture & MicroVM Isolation Standards 2026

Protocolos Abiertos: MCP y ReAct

Desacoplamiento arquitectónico mediante JSON-RPC 2.0 y el ciclo reflexivo Pensar-Actuar-Observar

El Ciclo ReAct en la Nube

1. Thought (Razonamiento):
Evalúa síntomas, consulta manuales RAG y define la siguiente hipótesis.

2. Action (Ejecución de Herramienta):
Invoca endpoints seguros con esquemas estrictos JSON-Schema tipados.

3. Observation (Lectura del Entorno):
Asimila métricas y logs reales del sistema operativo para iterar.

Model Context Protocol (MCP)

Desacoplamiento Absoluto:
El cliente habla con servidores de datos mediante JSON-RPC 2.0 estándar.

Primitivas Canónicas:
Resources (datos de solo lectura), Prompts (guías) y Tools (acciones).

Independencia de Proveedor:
Alterna entre Claude, OpenAI y DeepSeek sin tocar las herramientas.

Fuente: Model Context Protocol (MCP) Standard & ReAct Pattern Architecture 2026

Soberanía de Datos: Ley 1581 de Colombia

Cumplimiento normativo de Habeas Data, directrices de la SIC y anonimización de datos corporativos

1. Transferencia Internacional

Restricción Legal:

Prohibido transferir datos personales sin consentimiento expreso o país adecuado.

Solución Técnica:

Módulo de sanitización local con spaCy; enmascara cédulas, nombres y salarios.

Cumplimiento formal Ley 1581 / SIC

2. Residencia Obligatoria

Datos Altamente Sensibles:

Historias clínicas, telemetría crítica de plantas químicas o inteligencia judicial.

Solución Técnica:

Inferencia auto-alojada con vLLM sobre nodos OpenStack dentro de Colombia.

Cero salida hacia nubes foráneas

3. Retención Cero (ZDR)

Pacto Contractual en MaaS:

Garantía estricta de no persistencia en disco ni reutilización para entrenamiento.

Cifrado de Extremo a Fin:

Túneles TLS 1.3 vía VPC PrivateLink con llaves KMS bajo custodia del cliente.

Auditoría certificada SOC 2 / ISO 27001

Fuente: Ley Estatutaria 1581 de 2012 y Guías de Tratamiento de Datos Personales en IA · SIC Colombia

MATRIZ CONSOLIDADORA

Matriz de Arquitectura de la Unidad 3

Síntesis multidimensional de decisiones de infraestructura, cómputo y servicio aprendidas en el curso

Dimensión de Arquitectura	Nube Privada (OpenStack)	Inferencia IA (vLLM / AWQ)	Sist. Agénticos (MCP)
Tecnología Central	Nova, Neutron OVN, Cinder Ceph	vLLM, PagedAttention, INT4	Protocolo MCP, ReAct
Recurso Crítico Físico	Ancho de banda L2/L3 y Ceph IOPS	Ancho de banda HBM3 (3.35 TB/s)	Latencia RPC y Sandboxes
Mecanismo de Aislamiento	Hipervisores KVM y Geneve overlays	VPC Endpoints con ZDR cifrado	Firecracker y RBAC IAM
Regla de Optimización	Alta disponibilidad con 3 controladores	Cuantizar AWQ para ahorrar 75% VRAM	Poda y límites max_iter

Síntesis para el Examen y la Vida Profesional:

La arquitectura de nube moderna no aísla los componentes: integra cómputo virtualizado resiliente con aceleradores de silicio y agentes autónomos bajo un marco inquebrantable de seguridad, economía de recursos y soberanía de datos.

Fuente: Architecture Decision Framework · CUC Arquitectura en la Nube 2026-2

Conclusiones Finales y Próximos Desafíos

Agradecimiento y despedida de Arquitectura en la Nube · Universidad de la Costa (CUC)

Misión Cumplida · Arquitectura en la Nube 2026-2

Unidad 1: Fundamentos Cloud

Modelos IaaS, PaaS, SaaS

Virtualización, hipervisores KVM

Redes y almacenamiento en bloque

Fundamentos consolidados ✓

Unidad 2: Nube Pública AWS

Diseño VPC multizona resiliente

Cómputo elástico EC2 y autoescalado

Bases de datos Aurora y seguridad IAM

Diseño escalable dominado ✓

Unidad 3: Nube Privada e IA

OpenStack HA y redes OVN Geneve

Inferencia vLLM, AWQ y GPUs H100

Agentes MCP y Ley 1581 Colombia

Frontera tecnológica alcanzada ✓

¡Felicitaciones a todos los futuros ingenieros de sistemas de la CUC!

Llevan consigo las habilidades teóricas, matemáticas y de ingeniería para liderar la transformación digital del Caribe colombiano.

Éxitos en sus proyectos profesionales y nos vemos en las próximas etapas de su carrera.